

**Challenges and opportunities faced by computing educators and students adapting to LLMs capable of generating accurate source code from natural-language problem descriptions.**

BY PAUL DENNY, JAMES PRATHER, BRETT A. BECKER, JAMES FINNIE-ANSLEY, ARTO HELLAS, JUHO LEINONEN, ANDREW LUXTON-REILLY, BRENT N. REEVES, EDDIE ANTONIO SANTOS, AND SAMI SARSA

Computing Education in the Era of Generative AI

A NEW ERA is emerging in which artificial intelligence (AI) will play an ever-increasing role in many facets of daily life. One defining characteristic of this new era is the ease with which novel content can be generated. Large language models (LLMs)—neural network-based models trained on vast quantities of text data⁴—are capable of creating a variety of convincing human-like

outputs, including prose, poetry, and source code. It is largely accepted that synthesizing source code automatically from natural-language prompts is likely to improve the productivity of professional developers²⁶ and is being actively explored by well-funded entities such as OpenAI (ChatGPT, GPT-4^a), Amazon (CodeWhisperer^b), and Google (Alpha-Code,²¹ Bard^c). In the same way that high-level programming languages offered large productivity advantages over assembly-language programming many decades ago, AI code-generation tools appear primed to transform traditional programming practices. Claims are already emerging that a significant proportion of new code is being produced by tools such as GitHub Copilot,⁹ a plug-in for popular integrated development environments (IDEs), such as Visual Studio Code.

The current pace of development in this area is staggering, with noticeably more advanced versions being released several times per year. The pace of advancement is so rapid that in March 2023, a well-publicized open letter appeared that encour-

a See <https://openai.com/research/gpt-4>

b See <https://aws.amazon.com/codewhisperer>

c See blog.google/technology/ai/code-with-bard

» key insights

- **Generative AI presents challenges and opportunities for computing education, necessitating updated pedagogical strategies that focus on new skill sets.**
- **Generative AI models are highly capable of generating solutions to problems typical of introductory programming courses, raising concerns around potential student overreliance and misuse.**
- **AI-driven tools transform the creation and customization of educational resources such as programming exercises, enabling the efficient generation of personalized learning materials.**
- **Novel pedagogical approaches are emerging to teach students how to leverage generative AI, emphasizing strategic problem decomposition and the importance of accuracy when specifying programming tasks to AI systems.**



aged a public, verifiable, and immediate pause of at least six months on the training of AI systems more powerful than GPT-4. Signed by Elon Musk, Steve Wozniak, Moshe Vardi, and thousands of others including many AI leaders and Turing award recipients,^d the letter was addressed to all AI labs and suggested potential government-led moratoriums.

These developments raise urgent questions about the future direction of many aspects of society, including computing education. For example, one popular evidence-based pedagogy for teaching introductory programming involves students writing many small exercises that are checked either manually or by automated grading tools. However, these small problems can now easily be solved by AI models. Often, all that is required of a student is to accept an auto-generated suggestion by an IDE plug-in.^{10,11} This raises concerns that students may use new tools in ways that limit learning and make the work of educators more difficult. Bommasani et al. highlight that it has become much more complex for teachers "to understand the extent of a student's contribution" and "to regulate ineffective collaborations and detect plagiarism."⁴ Alongside such challenges emerge opportunities for students to learn computing skills.²

This article discusses the challenges and opportunities such models present to computing educators, with a focus on introductory programming classrooms. This discussion is organized around two foundational articles from the computing education literature written around the time that awareness of code-generating language models was just emerging. The first, published in Jan. 2022, evaluated the performance of code-generating models on typical introductory-level programming problems. The second, published in Aug. 2022, explored the quality and novelty of learning resources generated by these models. Now, we consider implications for computing education in light of new model capabilities and as lessons emerge from educators incorporating such models into their teaching practices.

Large language models and code.

Instructors should be extremely clear about when and how generative AI tools are allowed to be used on their assessments.

AI-driven coding has only been a viable reality for the general public since 2022, when GitHub's Copilot emerged from a period of technical preview. Originally pitched as "your AI pair programmer," at the time of writing, Copilot claims to be the "world's most widely adopted AI developer tool."^e Other AI-powered code-generation tools are also broadly accessible, including Amazon's CodeWhisperer and Google's Bard. The Codex model (discussed in this article specifically) was the original model to power Copilot. A descendant of GPT-3, Codex was fine-tuned with code from more than 50 million public GitHub repositories totaling 159GB.⁵ Although now officially deprecated in favor of the newer chat models, Codex was capable of taking English-language prompts and generating code in several programming and scripting languages, including JavaScript, Go, Perl, PHP, Python, Ruby, Swift, TypeScript, and shell. It could also translate code between programming languages, explain (in several natural languages) the functionality of code, and return the time complexity of the code it generated.

The use of such tools in education is nascent and changing rapidly. Copilot was only made freely available to students in June 2022^f and to teachers in September 2022^g after its potential to impact education began to unfold. In November 2022, ChatGPT^h was released, followed by the release of GPT-4 in March 2023. OpenAI has continued to update these models with new features, such as data analysis from files, analyzing images, and assisted Web search. For a more technical overview of the historical developments and future trends of language models, read the *Communications* article by Li.²⁰

Challenges Ahead

Code-generation tools powered by LLMs can correctly and reliably solve many programming problems that are typical in introductory courses. This raises a number of important

^d See <https://bit.ly/3tzJoHg>

^e See <https://github.com/features/copilot>

^f See <https://bit.ly/41FcEsz>

^g See <https://bit.ly/48euq8w>

^h See <https://openai.com/blog/chatgpt>

questions for educators. For example, just how good are these tools? Can a student with no programming knowledge, but who is armed with a code-generating LLM, pass typical programming assessments? Do we need a different approach?

Putting them to the test. To explore the performance of LLMs in the context of introductory programming, we prompted Codex with real exam questions and compared its performance to that of students taking the same exams. We also prompted Codex to solve several variants of a well-known CS1-level programming problem (the “Rainfall problem”) and examined the correctness and variety of solutions produced. This work was originally performed in September 2021, several weeks after OpenAI provided API access to the Codex model. The resulting paper, published in Jan. 2022, was the first in a computing education venue to assess the code-generating capabilities of LLMs.¹⁰

My AI wants to know if its grade will be rounded up. We took all questions from two Python CS1 programming exams that had already been taken by students and provided them as input (verbatim) to Codex. The exam questions involved common Python datatypes, including strings, tuples, lists, and dictionaries. They ranged in complexity from simple calculations, such as computing the sum of a series of simulated dice rolls, to more complex data manipulations, such as extracting a sorted list of the keys that are mapped to the maximum value in a dictionary.

To evaluate the code generated, we executed it against the same set of test cases that were used in assessing the student exams. This follows a similar evaluation approach employed by the Codex developers.⁵ If the Codex output differed from the expected output with only a trivial formatting error (for example, a missing comma or period) we made the appropriate correction, much as a student would if using Codex to complete an exam.

To contextualize the performance of the Codex model, we calculated the score for its responses in the same way as for real students using the same question weights and accumulated penalties for incorrect submis-

sions. Codex scored 15.7/20 (78.5%) on Exam 1 and 19.5/25 (78.0%) on Exam 2. Figure 1 plots the scores (scaled to a maximum of 100) of 71 students enrolled in the CS1 course in 2020 who completed both exams. Codex’s score is marked with a blue ‘X’. Averaging both Exam 1 and Exam 2 performance, Codex ranks 17 amongst the 71 students, placing it within the top quartile of class performance.

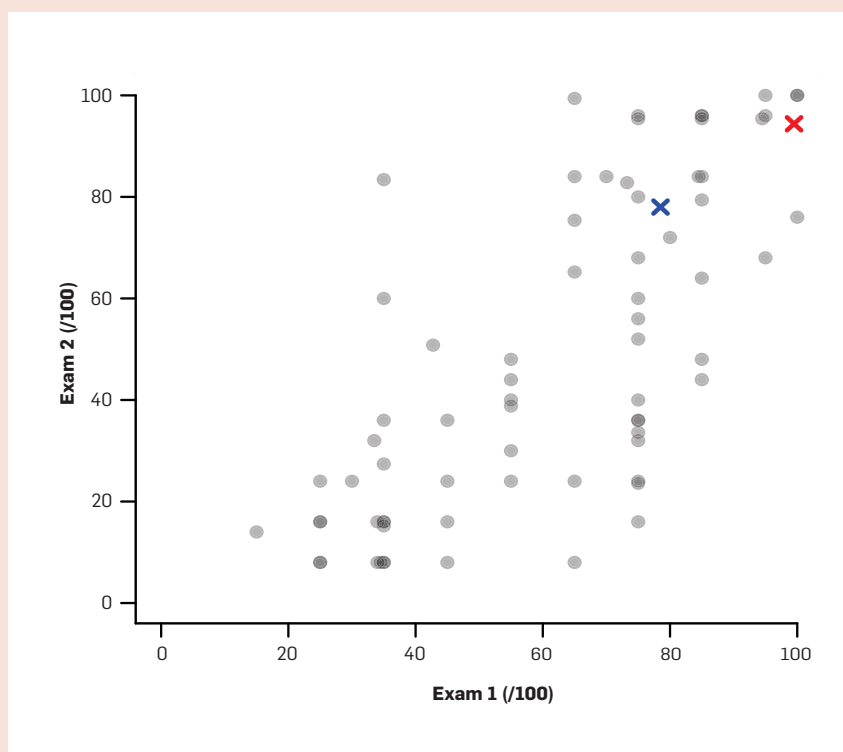
We observed that some of the Codex answers contained trivial formatting errors. We also observed that Codex performed poorly with problems that disallowed the use of certain language features (for example, using `split()` to tokenize a string). Codex often did not produce code that avoided using these restricted features, and thus the model (in these cases) often did not pass the auto-grader. Codex also performed poorly when asked to produce formatted ASCII output, such as patterns of characters forming geometric shapes, especially where the requirements were not specified in the problem description but had to be inferred from the provided ex-

ample inputs and outputs.

Yes, I definitely wrote this code myself. To understand the amount of variation in the responses, we provided Codex with seven variants of the problem description for the well-studied ‘Rainfall’ problem (which averages values in a collection) a total of 50 times each, generating 350 responses. Each response was executed against 10 test cases (a total of 3,500 evaluations). Across all variants, Codex had an average score close to 50%. Codex performed poorly on cases where no valid values were provided as input—for example, where the collection to be averaged was empty.

We also examined the number of source lines of code for all Rainfall variants, excluding blank and comment lines. In addition, we classified the general algorithmic approach employed in the solutions as an indicator of algorithmic variation. We found that Codex provides a diverse range of responses to the same input prompt. Depending on the prompt, the resulting programs used varied programmatic structures, while ultimately favoring expected methods for each problem variation—that is,

Figure 1. Student scores on Exam 1 and Exam 2, represented by circles. Codex’s 2021 score is represented by the blue ‘X’. GPT-4’s 2023 score on the same questions is represented by the red ‘X’.



for-loops for processing lists and while-loops for processing standard input.

Rapid progress. Given the improvement in model capabilities over the last two years, it is interesting to observe how well a state-of-the-art model (GPT-4 at the time of writing) performs on the same set of questions. In July 2023, a working group exploring LLMs in the context of computing education replicated this study using GPT-4 under identical conditions.²⁹ GPT-4 scored 99.5% on Exam 1 and 94.4% on Exam 2, this time outscored overall by only three of the 71 students (GPT-4 is represented by the red ‘X’ in Figure 1). On the Rainfall problems, GPT-4 successfully solved every variant, in some cases producing the correct result but with a trivial formatting error. Another follow-up study looked at the performance of generative AI on CS2 exam questions and found that it performed quite well in that context.¹¹ Newer models can also solve other types of programming exercises, like Parsons Problems, with decent accuracy that is likely to only improve over time.³¹

Academic integrity. Software development often encourages code reuse and collaborative development practices, which makes the concept of academic integrity difficult to formalize in computing.³⁵ Nevertheless, individual work is still commonplace in computing courses, and it is an expectation for students working on individual projects to produce their own code rather than copy code written by someone else. This is often verified through the use of traditional plagiarism tools. However, recent work has shown that common plagiarism detection tools are often ineffective against AI-generated solutions.³ This raises significant concerns for educators monitoring academic integrity in formal assessments.

Academic misconduct. Although academic misconduct has been discussed in the computing education community for quite some time,³⁵ the advent of LLMs provides a new and difficult set of challenges. The first is categorizing exactly what type of academic misconduct, if any, its usage falls into. A recent working-group report on LLMs in computing education considered ethics and examined it in the context of the ACM Code of Ethics and recent

university AI usage policies.²⁹ They discussed plagiarism, collusion, contract cheating, falsification, and the use of unauthorized resources. Though many university policies have placed AI usage into the category of plagiarism, Prather et al. disagree.²⁹ Plagiarism involves stealing content from a person with agency, which LLMs as next-token generators clearly do not have. If generative AI tools are seen as productivity tools (such as IDE code-completion or calculators for mathematical problems) that are used professionally, then it makes sense to decide if the use of such tools is appropriate for a given context and communicate the decision to students. If students persist in using the tools when they are restricted, then they would be engaging in academic misconduct because they used an unauthorized resource, not because of some intrinsic characteristic of the tool itself. Instructors should, therefore, be extremely clear about when and how generative AI tools are allowed to be used on their assessments. The working-group report includes a guide for students that could easily be adapted by faculty into a helpful hand-out or added to a course syllabus.

A recent interview study with computing educators has revealed that initial reactions are divided—from banning all use of generative AI to an acceptance that resistance is, ultimately, futile.¹⁷ Restricting the use of generative AI tools is likely (at least in the short term) to shift practice toward increased use of secure testing environments⁴³ and a greater focus on the development and assessment of process skills.¹⁶

Code reuse and licensing. Potential licensing issues arise when content is produced using code-generation models, even when model data is publicly available.²¹ Many different licenses apply to much of the publicly available code used to train LLMs, and typically these licenses require those who reuse to credit the code they used, even when the code is open source. When a developer generates code using an AI model, they may end up using code that requires license compliance without being aware of it. Such issues are already before the courts.ⁱ This is

i See <https://githubcopilotlitigation.com>

clearly an issue that extends beyond educational use of software, but as educators it is our role to inform students of their professional responsibilities when reusing code.

Learner over-reliance. The developers of Codex noted that a key risk of using code-generation models in practice is users’ over-reliance.⁵ Novices using such models, especially with tools such as Copilot that embed support in an IDE, may quickly become accustomed to auto-suggested solutions. This could have multiple negative effects on student learning.

Metacognition. Developing computational thinking skills is important for novice programmers as it can foster higher-order thinking and reflection.²³ Metacognition, or “thinking about thinking”, is a key aspect of computational thinking (and problem-solving in general). Learning to code is already a challenging process that requires a high level of cognitive effort to remember language syntax, think computationally, and understand domain-specific knowledge; the use of metacognitive knowledge and strategies can aid in problem-solving and prevent beginners from getting overwhelmed or lost. Relying too heavily on code-generation tools may hinder the development of these crucial metacognitive skills.

When the models fail. Despite encouraging results such as those presented here, an analysis of solutions generated by AlphaCode revealed that 11% of Python solutions were syntactically incorrect (produced a SyntaxError) and 35% of C++ solutions did not compile.²¹ Recent work has shown that as many as 20% of introductory programming problems are not solved sufficiently by code-generation models, even when allowing for expert modification of the natural-language problem descriptions.⁶ The developers of Codex noted that it can recommend syntactically incorrect code, including variables, functions, and attributes that are undefined or outside the scope of the codebase, stating, “Codex may suggest solutions that superficially appear correct but do not actually perform the task the user intended. This could particularly affect novice programmers and could have significant safety implications depending on the

context”.⁵ Students who have become overly reliant on model outputs may find it especially challenging to proceed when the suggested code is incorrect and cannot be resolved through natural-language prompting.¹⁵

Bias and bad habits. The issue of bias in AI is well known. In addition to general bias (subtle or overt) that affects nearly all AI-generated outputs, such as the representation of particular demographics and genders, there are also likely biases unique to AI code generation.

Appropriateness for beginners. Novices usually start by learning simple programming concepts and patterns, gradually building their skills. However, much of the vast quantity of code on which these AI models are trained was written by experienced developers. Therefore, we should expect that AI-generated code may sometimes be too advanced or complex for novices to understand and modify. Recent work has shown that even the latest generative AI models generate code using concepts too advanced for novices or that are specifically outside the curriculum.¹⁵

Harmful biases. The developers of Codex found that code-generation models raise bias and representation issues—notably that Codex can generate code comments (and potentially identifier names) that reflect negative stereotypes about gender and race, and may include other denigratory outputs.⁵ Such biases are obviously problematic, especially where novices are relying on the outputs for learning purposes. Notably, the feature list for Amazon CodeWhisperer includes capabilities to remove harmful biases from generated code.^j Some recent work (from competitor Microsoft) has expressed doubt about the reliability of this feature.³³

Security. Unsurprisingly, AI-generated code can be insecure,²⁵ and human oversight is required for the safe use of AI code-generation systems.⁵ However, novice programmers lack the knowledge to provide this oversight. A recent exploration of novices using AI code-generation tools found they consistently wrote insecure code with specific vulnerabilities in string



Developing computational thinking skills is important for novice programmers as it can foster higher-order thinking and reflection.



encryption and SQL injection.²⁷ Perhaps even more disturbing, the novice programmers in this study who had access to an AI code-generating tool were more likely to believe they had written secure code. This reveals a pressing need for increased student and educator awareness around the limitations of current models for generating secure code.

Computers in society. The use of AI-generated code provides many opportunities for discussions on ethics and the use of computers in society. Moreover, these technologies may serve as a vehicle to empower novice users to explore more advanced ideas earlier, leveraging the natural engagement that comes from using technologies that are “in the news.” Teachers of introductory courses have long told themselves that students will learn about testing, security, and other more advanced topics in subsequent courses. However, with growing numbers of students taking introductory classes but not majoring in computing, and the capabilities that code generation affords, the stakes are higher for CS1 and introductory classes to raise these issues early, before the chance of real-world harm is great.

Opportunities Ahead

Despite the challenges that must be navigated, code-generation tools have the potential to revolutionize teaching and learning in the field of computing.² Indeed, developers of such models specifically highlight their potential to positively impact education. When introducing Codex, Chen et al. outline a range of possible benefits, including to: “aid in education and exploration.”⁵ Similarly, the developers of AlphaCode suggest such tools have “the potential for a positive, transformative impact on society, with a wide range of applications including computer science education.”²¹ In this section, we discuss several concrete opportunities for code- and text-generation models to have a transformative effect on computing education.

Plentiful learning resources. Introductory programming courses typically use a wide variety of learning resources. For example, programming exercises are a very common

j See <https://go.aws/3NKd2AI>

type of resource for helping students practice writing code. Similarly, natural-language explanations of code are another useful resource. They can be valuable for helping students understand how a complex piece of code works or as a tool for evaluating student comprehension of code. However, it is a significant challenge for educators to generate a wide variety of high-quality exercises targeted to the interests of individual learners and to produce detailed explanations at different levels of abstraction for numerous code examples.

We explored the potential for LLMs to reduce the effort needed by instructors to generate the two types of learning resources just discussed: programming exercises and code explanations. This work, which was originally carried out in April 2022 and published in August 2022, was the first paper in a computing education venue to explore LLM-generated learning resources.³⁴

Programming exercises. Figure 2 shows an example of the input we used to generate new programming exercises using Codex. This ‘priming’ exercise consists of a one-shot example (a complete example similar to the desired output) followed by a partial prompt to prime the generation of a new output. In this case, the format of the priming exercise consists of a label (Exercise 1) followed by keywords for both the contextual themes (donuts) and the programming-related concepts (function, conditional) of the exercise, a natural-language problem statement and a solution (in the form of a Python function). For space reasons, we omit a list of test cases, but these can also be included for programming problems. The priming input ends with the explicit prompt for a new exercise to be generated (Exercise 2), along with the desired concepts and themes expressed as keywords (basketball, function, list, and for loop).

Figure 3 shows one output generated when the prompt in Figure 2 was provided to Codex. In this case, as requested by the keyword information in the input prompt, the problem statement is related to basketball and the model solution consists of a *function* that involves a *list* and a *for loop*. To evaluate this approach more thor-



The use of AI-generated code provides many opportunities for discussions on ethics and the use of computers in society.



oughly, we generated a set of 240 programming exercises by varying the programming-related concepts and contextual themes. We attempted to execute the generated code against the generated test cases and analyzed statement coverage as a measure of the thoroughness of the test suite. Table 1 summarizes these results and shows that in most cases, the programming exercises generated by the model included sample solutions that were executable. Similarly, most of the time, the model also generated a set of tests, resulting in a total of 165 programming exercises with both a sample solution and a set of tests. The sample solution frequently did not pass all the generated tests, but when it did, the test suites achieved full statement coverage in all but three cases.

We also found that the vast majority of exercises (around 80%) were entirely novel, in that fragments of the problem descriptions were not indexed by any search engines. A similar fraction of the exercises also matched the desired topics and themes. Although this is far from perfect, there is obvious potential for generating new and useful resources in this manner and the cost of eliminating poor results (which could be automated) is almost certainly smaller than manually generating a large number of exercises and accompanying test cases. With the addition of filtering steps that could be automated, it would be possible to generate an almost endless supply of novel resources that are contextualized to students’ interests.

Code explanations. Code explanations can be generated at different levels of abstraction, from high-level summaries to detailed explanations of every line. We focused on the latter, as these are often useful for students when debugging code. We prompted Codex using a simple input that consisted of the source code to be explained, the text “Step-by-step explanation of the above program”, and finally “1.” to influence the output to include numbered lines. We analyzed the resulting explanations in terms of completeness and accuracy, finding that 90% of the explanations covered all parts of the code, and nearly 70% of the explanations for individual lines were correct. Common errors were mostly related to relational operators

and branching conditionals—for example, where Codex stated “less than or equal to x” when the corresponding code was checking “less than x”.

Rapid progress. In this section, we described early work in which code explanations were generated using a version of the Codex model that was available in early 2022 (specifically, ‘code-davinci-001’). Less than a year later, code explanations generated by models such as ChatGPT are considerably better and more consistently accurate. Figure 4 illustrates one example of a code explanation generated by ChatGPT when provided only the code shown in the “Sample solution” area in Figure 3 and using the same prompt for a line-by-line explanation as described in this section.

The quality of LLM-generated learning resources is likely to continue improving alongside model capabilities. For example, MacNeil et al. found that code explanations generated by the GPT-3 model were consistently more helpful than those generated by Codex.²⁴ They generated several different kinds of code explanations, deploying them in an online interactive e-book, and found that students reported high-level summaries of code as being more useful for their learning compared to lower-level detailed explanations of each line. Recent work has also found that LLM-created code explanations are rated more highly on average by students than code explanations created by their peers.¹⁸

We see great potential for LLMs to be applied to the production of a variety of learning resources relevant to computing education. We also expect the quality and accuracy of the generated resources to improve considerably over the near term based on recent trends.

Better programming error messages. For more than 60 years, researchers have identified poor programming error messages (PEMs) as problematic, and significant work remains in this area. Recent work has attempted to put error messages into more natural language by focusing on readability, which has been shown to improve student understanding of error messages and the number of successful code corrections.⁸ While it is

clear that increasing the readability of PEMs is helpful to novices, doing so at scale, and across languages, remains a challenge.

Leinonen et al. explored the potential of LLMs for improving PEMs.¹⁹ They collected Python error messages that had been reported as most unreadable in prior work and generated code examples that produced these error messages. They prompted the Codex API with both the code and er-

ror message to generate explanations of the PEMs and actionable fixes. They found that most of the explanations created by Codex were comprehensible, and Codex produced an output with an explanation for most inputs.

More recent work has extended this approach by implementing GPT-enhanced LLM explanations of PEMs directly into compilers or automated assessment tools. Taylor et al. deployed GPT explanations to a C/C++

Figure 2. A ‘priming’ exercise consisting of one complete example followed by the prompt for a new one.

```
"""Exercise 1
--Keywords--
donuts
function
conditional
--Problem statement--
Write a function called donut_cost that takes
↳ three inputs: the price of a donut, the
↳ number of donuts bought, and whether or not
↳ it is the weekend. The function should
↳ return the total cost of the donuts. If it
↳ is the weekend, the donuts cost twice as
↳ much.
--Sample solution--
def donut_cost(price, number, is_weekend):
    if is_weekend:
        return price * number * 2
    else:
        return price * number
"""Exercise 2
--Keywords--
basketball
function
list
for loop
--Problem statement--
```

Figure 3. Example output generated by Codex using the priming exercise from Figure 2.

```
Write a function called count_rebounds that
↳ takes a list of basketball players as an
↳ input. The function should return the
↳ total number of rebounds for the entire
↳ team. Each element in the list is itself a
↳ list containing the player's name, their
↳ points, and their rebounds.
--Sample solution--
def count_rebounds(players):
    total = 0
    for player in players:
        total = total + player[2]
    return total
```

compiler in CS1 and CS2 courses and found it provided accurate explanations in 90% of cases for compile-time errors and 75% of cases for run-time errors.³⁷ Wang et al. found that students receiving GPT-enhanced PEMs in a large-scale introductory programming course repeated an error 23.5% less often and resolved an error in 36.1% fewer attempts.⁴⁰ While there is still work to be done before the decades-old problem is solved, the potential to demystify PEMs in this way is an exciting opportunity only recently made possible.

Table 1. Analysis of 240 generated programming exercises, showing the proportion that met criteria regarding sample solutions and test cases.

Has sample solution?	203/240	84.6%
Sample solution executable?	182/203	89.7%
Has test cases?	170/240	70.8%
All tests pass?	51/165	30.9%
Full (100%) statement coverage?	48/51	94.1%

Exemplar solutions. Students often seek exemplar solutions when coding, either to check against their own code or to get help when struggling. However, instructors may not have the time to provide model solutions for every exercise, including historical test and exam questions. AI-generated code offers a time-saving alternative, with the ability to produce a variety of solutions which can help students understand and appreciate different tradeoffs in problem-solving, as suggested by Thompson et al.³⁸

The ability to generate exemplar solutions automatically can shift the emphasis from just ensuring that code is correct to focusing on the differences between multiple correct solutions, and the need to make judgments on code style and quality. Extensive research on the benefits of peer review of code¹² suggests it is beneficial to consider multiple solutions to a problem, even if some are flawed. Code-generation models can be used to create solutions of varying quality, and these can be used for as-

essment tasks that require students to apply the critical analysis skills needed for code evaluation. This can facilitate discussions about different approaches and the quality of solutions, and provide opportunities for refactoring exercises.¹⁰

New pedagogical approaches. Computing educators are still working through the implications of LLMs in their classrooms, and a consensus about how to update pedagogy has yet to form. However, some early approaches are emerging.

LLMs early. In a traditional CS1 course, the initial focus usually begins with syntax and basic programming principles, and it can take time for students to become proficient in these fundamentals. One novel approach for progressing more rapidly to complex problems is to teach students how to use LLMs to handle low-level implementation details. This is exemplified by the approach in the textbook by Zingaro and Porter, “*Learn AI-Assisted Python Programming: With GitHub Copilot and ChatGPT*”.²⁸ Students are introduced to the GitHub Copilot plug-in within the Visual Studio Code IDE before they have learned to write a single line of Python code. A top-down approach is followed, where students decompose larger projects into smaller functions that are then solvable using Copilot by providing natural-language comments. This textbook provides a blueprint for how introductory courses could initially concentrate more on problem-solving and algorithms, rely on automatic code generation for implementation, and defer in-depth and nuanced discussions of syntax until later.

Explaining algorithmic concepts clearly. It is well known that the outputs produced by LLMs are very sensitive to their inputs.³² In fact, “prompt engineering,” where effective prompts are crafted, has emerged as a distinct (and nascent) skill when working with these models. For example, when using Codex to solve probability and statistics problems, engineering the prompt to include explicit hints on the strategy for solving a problem is extremely effective.³⁶ Denny et al. found that prompt engineering strategies, which described algorithmic steps, were effective for solving pro-

Figure 4. Explanation generated by the ChatGPT model of the code shown in the ‘Sample solution’ area in Figure 3.



Certainly! Here's a step-by-step explanation of the program:

1. The function '**count_rebounds**' takes in a list of players as an argument. Each player is represented as a tuple containing three elements: a player's name (a string), their number of points scored (an integer), and their number of rebounds (also an integer).
2. The function initializes a variable '**total**' to 0. This variable will be used to store the total number of rebounds among all the players.
3. The function enters a '**for**' loop that iterates over each player in the list of players.
4. For each player, the function adds the player's number of rebound to the '**total**' variable.
5. After the '**for**' loop completes, the function returns the value of the '**total**' variable. This is the total number of rebounds among all the players.

gramming tasks for which Copilot initially generated incorrect solutions.⁶ Other recent work has shown that developers are more successful working with Copilot when they decompose larger programming statements into smaller tasks and then explicitly prompt Copilot for each of the sub-tasks.^{1,13} It is likely that students will need to develop new skills to communicate effectively with these models. A key skill will be the ability to describe the computational steps they wish to achieve in natural language as a way of guiding the model to produce valid outputs.

Specification-focused tasks. One way for students to learn how to create effective prompts is to focus on writing task specifications. In a traditional introductory course, novices are presented with problem statements that have been very carefully specified by the instructor to be clear and unambiguous. Such detailed specifications provide excellent context for code-generation models to produce correct code solutions. New types of problems could task students with generating clear specifications themselves, and thus strengthen skills around LLM prompting. For example, this is the goal of ‘Prompt Problems’,⁷ in which students are presented with a visual representation of a problem that illustrates how input values should be transformed to an output. Their task is to devise a prompt that would guide an LLM to generate the code required to solve the problem. Prompt-generated code is evaluated automatically and can be refined iteratively until it successfully solves the problem. Recent work investigating classroom use of Prompt Problems has shown that students find them useful for strengthening their computational thinking skills and exposing them to new programming constructs.

A focus on refactoring. Students sometimes experience difficulty getting started on programming assignments, sometimes referred to as the programmer’s writer’s block. Recent work found that Copilot can help students overcome this barrier by immediately providing starter code, enabling them to build upon existing code rather than starting from scratch with a blank code editor.³⁹ This ap-



Students reported high-level summaries of code as being more useful for their learning compared to lower-level detailed explanations of each line.



proach may require a shift in focus toward tasks such as rewriting, refactoring, and debugging code, but it provides the opportunity to help students maintain momentum in a realistic setting where the ability to evaluate, rewrite, and extend code is often more important than writing every line of code from scratch.

Designing LLM tools. Programmers around the world, not just novices, will be using code generators in an increasing capacity moving forward. Exploring the integration of LLMs directly into educational environments, such as auto-graders and online textbooks, will be an important area of research. There is a need in such environments for appropriate guardrails so that generated outputs usefully support learning, without immediately revealing solutions or overwhelming novices with the complexity or quantity of feedback. Indeed, the announcement of GPT-4^k highlighted the example of a ‘Socratic tutor’ that would respond to a student’s requests with probing questions rather than revealing answers directly. One example of this integration in computing education is the work of Liffiton et al. on CodeHelp, an LLM-powered tool that uses prompt-based guardrails to provide programming students with real-time help but without directly revealing code solutions.²²

In general, adapting the feedback generated by LLMs to maximize learning in educational environments is likely to be an important research focus in the near future. Concrete recommendations are already beginning to emerge from very recent work in this space. First, the utilization of code generators by novices will generally decrease the number of errors they see. This seems like a positive experience, though it appears they are ill-equipped to deal with the errors they do see when presented with them.¹⁴ This means that tools must be designed to help users (of all skill levels) through the error-feedback loop. Second, generating and inserting large blocks of code may be counterproductive for users at all levels. This requires users to read through code they did not write, sometimes at a

^k See <https://openai.com/research/gpt-4>

more sophisticated level than they are familiar with. Novices may be intimidated by such code generation¹⁴ or may spend too much time reading code that does not further their goals.³⁰ Therefore, AI code generators should include a way for users to control the amount of code insertion and to specify how to step through a multi-part segment of generated code. Third, the fact that AI code generators are black boxes means that programmers of all skill levels may struggle to create correct mental models of how they work, which could harm their ability to fully utilize them or learn from their outputs. Explainable AI (XAI) patterns could be helpful here, such as exposing to the user a confidence value and user skill estimation above the generated code suggestion.³⁰ These suggestions are only the beginning of a new avenue of research on how to helpfully design usable AI code generators that empower novice learners and enhance programmer productivity.

Where Do We Go from Here?

The emergence of powerful code-generation models has led to speculation about the future of the computing discipline. In a recent *Communications* Opinion article, Welsh claims they herald the “end of programming” and believes there is major upheaval ahead for which few are prepared, as the vast majority of classic computer science will become irrelevant.⁴¹ In an even more recent article on BLOG@CACM, Meyer is equally impressed by the breakthroughs, placing them alongside the World Wide Web and object-oriented programming as a once-in-a-generation technology, but takes a more optimistic view.¹ In fact, Meyer predicts a resurgence in the need for classic software-engineering skills, such as requirements analysis, formulating precise specifications, and software verification.

Although the impact of generative AI tools is already evident for software developers, the long-term changes for computing education are less clear. Experts appreciate this new technology only because they already understand the underlying fundamentals. The ability to quickly generate large

Computing educators are still working through the implications of LLMs in their classrooms.

amounts of code does not eliminate the need to understand, modify, and debug code. Instead, it highlights how important it is to develop these basic competencies. Code literacy skills are essential to critically analyze what is being produced to ensure alignment between one's intentions and the generated code. Without the skills to read, test, and verify that code does what is intended, users risk becoming mere consumers of the generated content, relying on blind faith more than developed expertise. We argue that writing code remains a valuable way for novices to learn the fundamental concepts essential for code literacy.

Although future professional developers may indeed spend less time writing ‘low-level’ code, we believe generated code will still need to be modified and integrated into larger programs. We do expect to see some shift in emphasis, even in introductory courses, toward modifying code generated by AI tools, but the ability to edit such outputs and compose code in today's high-level languages will likely remain a fundamental skill for computing students. This aligns with Yellin's recent viewpoint that as programs increase in complexity, natural language becomes too imprecise an instrument with which to specify them.⁴² At some point, editing code directly is more effective than issuing clarifying instructions in natural language.

Harnessed correctly, tools such as Copilot and ChatGPT have the potential to be valuable assistants for this learning. We see these tools as serving a valuable teaching support role: to explain concepts to a broad and diverse range of learners, generate exemplar code to illustrate those concepts, and generate useful learning resources that are contextualized to the interests of individuals. We also anticipate the emergence of new pedagogies that leverage code-generation tools, including explicit teaching of effective ways to communicate with the tools, and tasks that focus on problem specification rather than implementation.

In light of the rapid adoption of generative AI tools, it is essential that educators evolve their teaching methods and approaches to assessment. Curricula should also expand to cover the broader societal impact of generative

¹ See <https://bit.ly/3TXEJKb>

AI, including pertinent legal, ethical, and economic issues. We believe it is imperative to get ahead of the use of these tools, incorporate them into our classrooms from the very beginning, and teach students to use them responsibly. In short, we must embrace these changes or face being left behind. Embracing this shift is not just essential—it represents a chance to invigorate our educational practices. 

References

1. Barke, S., James, M.B., and Polikarpova, N. Grounded Copilot: How programmers interact with code-generating models. In *Proceedings of the ACM on Programming Languages* 7, 78 (2022), 85–111.
2. Becker, B.A. et al. Programming is hard – Or at least it used to be: Educational opportunities and challenges of AI code generation. In *Proceedings of the 54th SIGCSE Technical Symp. on Computer Science Education* 1, Association for Computing Machinery (2023), 500–506.
3. Biderman, S. and Raff, E. Fooling MOSS detection with pretrained language models. In *Proceedings of the 31st ACM Intern. Conf. on Information & Knowledge Management (CIKM '22)*, Association for Computing Machinery (2022), 2933–2943; 10.1145/3511808.3557079
4. Bommasani, R. et al. *On the opportunities and risks of foundation models*; 10.48550/ARXIV.2108.07258
5. Chen, M. et al. Evaluating large language models trained on code. (2021); <https://arxiv.org/abs/2107.03374>.
6. Denny, P., Kumar, V. and Giacaman, N. Conversing with Copilot: Exploring prompt engineering for solving CS1 problems using natural language. In *Proceedings of the 54th ACM Technical Symp. on Computer Science Education* 1, Association for Computing Machinery (2023), 1136–1142; 10.1145/3545945.3569823
7. Denny, P. et al. Prompt Problems: A new programming exercise for the generative AI era. In *Proceedings of the 55th ACM Technical Symp. on Computer Science Education* 1, Association for Computing Machinery (2024), 7; 10.1145/3626252.3630909
8. Denny, P. et al. On designing programming error messages for novices: Readability and its constituent factors. In *Proceedings of the 2021 CHI Conf. on Human Factors in Computing Systems*, Association for Computing Machinery, Article 55; 10.1145/3411764.3445696
9. Dohmke, T. GitHub Copilot for business is now available. *GitHub Blog*, (2023); <https://bit.ly/3twYP30>.
10. Finnie-Ansley, J., Denny, P., Becker, B.A., and Luxton-Reilly, A. The robots are coming: Exploring the implications of OpenAI Codex on introductory programming. In *Proceedings of the Australasian Computing Education Conf.*, Association for Computing Machinery (2022), 10–19; 10.1145/3511861.3511863
11. Finnie-Ansley, J. et al. My AI wants to know if this will be on the exam: Testing OpenAI's Codex on CS2 programming exercises. In *Proceedings of the 25th Australasian Computing Education Conf.*, Association for Computing Machinery, (2023), 97–104; 10.1145/3576123.3576134
12. Indriasari, T.D., Luxton-Reilly, A., and Denny, P. A review of peer code review in higher education. *ACM Trans. Comput. Education* 20, 3 (2020);
13. Jiang, E. et al. Discovering the syntax and strategies of natural language programming with generative language models. In *Proceedings of the 2022 CHI Conf. on Human Factors in Computing Systems*, Association for Computing Machinery, Article 386; 10.1145/3491102.3501870
14. Kazemitabaar, M. et al. Studying the effect of AI code generators on supporting novice learners in introductory programming. In *Proceedings of the 2023 CHI Conf. on Human Factors in Computing Systems*, Association for Computing Machinery, 23; 10.1145/3544548.3580919.
15. Kazemitabaar, M. et al. How novices use LLM-based code generators to solve CS1 coding tasks in a self-paced learning environment. In *Proceedings of the 23rd Koli Calling Conf. on Computing Education Research* (2023), 10.
16. Kussmaul, C. Process oriented guided inquiry learning (POGIL) for computer science. In *Proceedings of the 43rd ACM Technical Symp. on Computer Science Education*, Association for Computing Machinery (2012), 373–378; 10.1145/2157136.2157246
17. Lau, S. and Guo, P. From "ban it till we understand it" to "resistance is futile": How university programming instructors plan to adapt as more students use AI code generation and explanation tools such as ChatGPT and GitHub Copilot. In *Proceedings of the 2023 ACM Conf. on Intern. Computing Education Research* 1, Association for Computing Machinery, (2023), 106–121; 10.1145/3568813.3600138
18. Leinonen, J. et al. Comparing code explanations created by students and large language models. In *Proceedings of the 2023 Conf. on Innovation and Technology in Computer Science Education* 1, Association of Computing Machinery, 124–130; 10.1145/3587102.3588785.
19. Leinonen, J. et al. Using large language models to enhance programming error messages. In *Proceedings of the 2023 ACM SIGCSE Technical Symp. on Computer Science Education*; 10.1145/3545945.3569770
20. Li, H. Language models: Past, present, and future. *Communications of the ACM* 65, 7 (July 2022), 56–63; 10.1145/3490443
21. Li, Y. et al. Competition-level code generation with AlphaCode. *Science* 378, 6624 (2022), 1092–1097; 10.1126/science.abq1158
22. Liffiton, M., Sheese, B., Savelka, J., and Denny, P. CodeHelp: Using large language models with guardrails for scalable support in programming classes. In *Proceedings of the 23rd Koli Calling Intern. Conf. on Computing Education Research*, Association for Computing Machinery (2023); 10.1145/3631802.3631830
23. Loksa, D. et al. Metacognition and self-regulation in programming education: Theories and exemplars of use. *ACM Trans. Computing Education* 22, 4, Article 39 (Sept. 2022), 31; 10.1145/3487050
24. MacNeil, S. et al. Experiences from using code explanations generated by large language models in a Web software development e-book. In *Proceedings of the 54th ACM Technical Symp. on Computer Science Education* 1, Association for Computing Machinery (2023), 931–937; 10.1145/3545945.3569785
25. Pearce, H. et al. Asleep at the keyboard? Assessing the security of GitHub Copilot's code contributions. In *2022 IEEE Symp. on Security and Privacy*, 754–768; 10.1109/SP46214.2022.9833571
26. Peng, S., Kalliamvakou, E., Cihon, P., and Demirer, M. The impact of AI on developer productivity: Evidence from GitHub Copilot (2023), [arXiv:cs.SE/2302.06590](https://arxiv.org/abs/2302.06590)
27. Perry, N., Srivastava, M., Kumar, D., and Boneh, D. Do users write more insecure code with AI assistants? In *Proceedings of the 2023 ACM SIGSAC Conf. on Computer and Communications Security*, Association for Computing Machinery, 2785–2799; 10.1145/3576915.3623157
28. Porter, L. and Zingaro, D. *Learn AI-Assisted Python Programming With GitHub Copilot and ChatGPT*. Manning, Shelter Island, NY, USA (2023); <https://bit.ly/4aE7CRa>
29. Prather, J. et al. The robots are here: Navigating the generative AI revolution in computing education. In *Proceedings of the 2023 Working Group Reports on Innovation and Technology in Computer Science Education*, Association for Computing Machinery; 10.1145/3623762.3633499
30. Prather, J. et al. "It's weird that it knows what I want": Usability and interactions with Copilot for novice programmers. *ACM Trans. Comput.-Hum. Interact.* (Aug. 2023); 10.1145/3617367.
31. Reeves, B. et al. Evaluating the performance of code generation models for solving Parsons problems with small prompt variations. In *Proceedings of the 2023 Conf. on Innovation and Technology in Computer Science Education* 1, Association for Computing Machinery, 299–305; 10.1145/3587102.3588805
32. Reynolds, L. and McDonnell, K. Prompt programming for large language models: Beyond the few-shot paradigm. In *Extended Abstracts of the 2021 CHI Conf. on Human Factors in Computing Systems*, Association for Computing Machinery, 7; 10.1145/3411763.3451760
33. Sarkar, A. et al. What is it like to program with artificial intelligence? (2022); [arXiv:cs.HC/2208.06213](https://arxiv.org/abs/2208.06213)
34. Sarsa, S., Denny, P., Hellas, A., and Leinonen, J. Automatic generation of programming exercises and code explanations using large language models. In *Proceedings of the 2022 ACM Conf. on Intern. Computing Education Research* 1, Association for Computing Machinery, 27–43; 10.1145/3501385.3543957
35. Simon et al. Negotiating the maze of academic integrity in computing education. In *Proceedings of the 2016 ITICSE Working Group Reports*, Association for Computing Machinery, 57–80; 10.1145/3024906.3024910
36. Tang, L. et al. Solving probability and statistics problems by probabilistic program synthesis at human level and predicting solvability. In *Artificial Intelligence in Education. Posters and Late Breaking Results, Workshops and Tutorials, Industry and Innovation Tracks, Practitioners' and Doctoral Consortium*, M.M. Rodrigo, N. Matsuda, A.I. Cristea, and V. Dimitrova (eds), Springer Intern. Publishing (2022), 612–615.
37. Taylor, A., Vassar, A., Renzella, J. and Pearce, H. Dcc –help: Transforming the Role of the Compiler by Generating Context-Aware Error Explanations with Large Language Models. In *Proceedings of the 55th ACM Technical Symposium on Computer Science Education V.1 (SIGCSE 2024)*, Association for Computing Machinery, New York, NY, USA, 1314–1320.
38. Thompson, E., Whalley, J., Lister, R., and Simon, B. Code classification as a learning and assessment exercise for novice programmers. In *Proceedings of the 19th Annual Conf. of the National Advisory Committee on Computing Qualifications* (2006), 291–298.
39. Vaithilingam, P., Zhang, T., and Glassman, E.L. Expectation vs. experience: Evaluating the usability of code generation tools powered by large language models. In *CHI Conf. on Human Factors in Computing Systems Extended Abstracts*, Association for Computing Machinery (2022), 1–7.
40. Wang, S., Piech, C., and Mitchell, J.C. A large scale RCT on effective error messages in CS1. In *Proceedings of the 55th ACM Technical Symp. on Computer Science Education* 1, Association for Computing Machinery (2023), 7.
41. Welsh, M. The end of programming. *Communications of the ACM* 66, 1 (Jan. 2023), 34–35; 10.1145/3570220
42. Yellin, D.M. The premature obituary of programming. *Communications of the ACM* 66, 2 (Feb. 2023), 41–44; 10.1145/3555367
43. Zilles, C.B., West, M., Herman, G.L., and Brett, T. Every university should have a computer-based testing facility. In *Proceedings of the 11th Intern. Conf. on Computer Supported Education*, I. H. Chad Lane, S., Zvacek, and J. Uhomoihi (eds), SciTePress (2019), 414–420; 10.5220/0007753304140420

Paul Denny (paul@cs.auckland.ac.nz) is an associate professor at the University of Auckland, Auckland, New Zealand.

James Prather is an associate professor at Abilene Christian University, Abilene, TX, USA.

Brett A. Becker is an assistant professor at University College Dublin, Dublin, Ireland.

James Finnie-Ansley is a Ph.D. student at the University of Auckland, Auckland, New Zealand.

Arto Hellas is a senior university lecturer at Aalto University, Espoo, Finland.

Juho Leinonen is a postdoctoral fellow at the University of Auckland, Auckland, New Zealand.

Andrew Luxton-Reilly is a professor at the University of Auckland, Auckland, New Zealand.

Brent N. Reeves is an associate professor at Abilene Christian University, Abilene, TX, USA.

Eddie Antonio Santos is a Ph.D. student at University College Dublin, Dublin, Ireland.

Sami Sarsa is a doctoral researcher at Aalto University, Espoo, Finland.

© 2024 Copyright held by the owner/author(s).